

Pruebas No Funcionales, Tipos de Pruebas, End to End y Cypress

Nombre del autor

 **Tiempo estimado: 10 min**

Universidad San Carlos de Guatemala
Facultad de Ingeniería

Escuela de Ingeniería en Ciencias y Sistemas.

Segundo Semestre 2026

[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Excelencia Técnica	Implementamos pruebas rigurosas que garantizan la calidad del software, verificando no solo que funcione sino que cumpla con estándares de rendimiento, seguridad y usabilidad
Responsabilidad	Cada desarrollador es responsable de escribir pruebas para su código, asegurando que los cambios no rompan funcionalidades existentes
Rigurosidad	Aplicamos metodologías sistemáticas de testing, documentando casos de prueba y resultados de manera precisa y verificable
Innovación	Utilizamos herramientas modernas como Cypress para automatizar pruebas, mejorando continuamente nuestros procesos de aseguramiento de calidad

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	Genera soluciones de software aplicando buenas prácticas de análisis, control de versiones y metodologías ágiles para garantizar proyectos eficientes y colaborativos en entornos virtuales de trabajo
Competencia Específica	Aplica técnicas de aseguramiento integral del software mediante la implementación de pruebas no funcionales, end-to-end y el uso de herramientas como Cypress para validar la calidad, rendimiento y confiabilidad de las aplicaciones

2. Palabras Clave

Pruebas No Funcionales: Evaluaciones que verifican atributos de calidad del sistema como rendimiento, seguridad, usabilidad y escalabilidad, más allá de la funcionalidad básica.

Tipos de Pruebas: Clasificación de pruebas según su objetivo (unitarias, integración, sistema, aceptación) y enfoque (funcionales vs no funcionales).

Pruebas End to End (E2E): Validación completa del flujo de una aplicación desde la perspectiva del usuario, simulando interacciones reales en un entorno similar a producción.

Cypress: Herramienta moderna de testing end-to-end para aplicaciones web que permite escribir, ejecutar y depurar pruebas de manera rápida y confiable.

Rendimiento: Capacidad del sistema para responder bajo carga de trabajo específica (tiempo de respuesta, throughput).

Escalabilidad: Habilidad del sistema para manejar crecimiento en usuarios, datos o transacciones sin degradación del servicio.

Automatización de Pruebas: Uso de software especializado para ejecutar casos de prueba de forma repetitiva y consistente.

3. Introducción

El aseguramiento integral del software es un pilar fundamental en el desarrollo de aplicaciones de calidad. Mientras que las pruebas funcionales verifican que el sistema haga lo que debe hacer, las pruebas no funcionales evalúan **cómo** lo hace: ¿es rápido?, ¿es seguro?, ¿puede escalar?, ¿es usable? En esta lectura, exploraremos los diferentes tipos de pruebas no funcionales, las estrategias de testing end-to-end y el uso de Cypress como herramienta moderna para automatizar la validación de aplicaciones web, proporcionando un enfoque completo para garantizar software robusto, eficiente y confiable.

4. Contenidos

Aseguramiento Integral del Software

Pruebas No Funcionales

Tabla 1 - Tipos de Pruebas No Funcionales

Tipo de Prueba	Qué Evalúa	Métricas Clave	Herramientas Comunes
Rendimiento	Tiempo de respuesta bajo carga	Latencia, throughput, uso de CPU/memoria	JMeter, LoadRunner, k6
Escalabilidad	Capacidad de crecer sin degradación	Usuarios concurrentes, transacciones/segundo	Gatling, Locust
Seguridad	Vulnerabilidades y protección de datos	OWASP Top 10, cifrado, autenticación	OWASP ZAP, Burp Suite
Usabilidad	Facilidad de uso y experiencia de usuario	Tasa de éxito, tiempo de aprendizaje, satisfacción	UserTesting, Hotjar
Compatibilidad	Funcionamiento en diferentes entornos	Navegadores, SO, dispositivos soportados	BrowserStack, Sauce Labs
Confiabilidad	Estabilidad y recuperación ante fallos	MTBF (tiempo medio entre fallos), disponibilidad	Chaos Monkey
Mantenibilidad	Facilidad de modificación y corrección	Complejidad ciclomática, deuda técnica	SonarQube

Figura 1 - Pirámide de Testing



Subtema: Tipos de Pruebas en el Ciclo de Desarrollo

Clasificación por Nivel:

1. **Pruebas Unitarias:** Validan componentes individuales (funciones, clases) de forma aislada.
2. **Pruebas de Integración:** Verifican la interacción entre módulos o servicios.
3. **Pruebas de Sistema:** Evalúan el sistema completo e integrado.
4. **Pruebas de Aceptación:** Validan que el software cumple con los requisitos del negocio.

Clasificación por Enfoque:

- **Caja Negra:** Sin conocimiento del código interno (perspectiva del usuario).
- **Caja Blanca:** Con conocimiento del código interno (perspectiva del desarrollador).
- **Caja Gris:** Combinación de ambos enfoques.

Subtema: Pruebas End to End (E2E)

Las pruebas E2E simulan el comportamiento real del usuario, validando flujos completos de la aplicación desde el frontend hasta el backend y bases de datos.

Características de las Pruebas E2E:

- Validan flujos críticos de negocio
- Detectan problemas de integración entre componentes
- Simulan interacciones reales del usuario
- Proporcionan confianza en releases
- Son más lentas y costosas de mantener
- Pueden ser frágiles ante cambios en la UI

Cuándo usar E2E:

- Flujos de autenticación y autorización
- Procesos de compra o transacciones críticas
- Integraciones con servicios externos
- Validación de flujos multi-paso

Subtema: Cypress - Herramienta Moderna de Testing

¿Qué es Cypress? Cypress es un framework de testing end-to-end diseñado para aplicaciones web modernas. A diferencia de herramientas tradicionales como Selenium, Cypress se ejecuta dentro del navegador, permitiendo un control total de la aplicación.

Ventajas de Cypress:

1. **Configuración sencilla:** No requiere drivers ni configuraciones complejas
2. **Tiempo real:** Visualización instantánea de pruebas mientras se ejecutan
3. **Debugging integrado:** Herramientas de desarrollo del navegador disponibles
4. **Espera automática:** No necesita waits o sleeps manuales
5. **Screenshots y videos:** Captura automática de fallos
6. **Red de tráfico controlada:** Permite mockear respuestas del servidor

Tabla 2 - Comparativa: Cypress vs Selenium

Característica	Cypress	Selenium
Arquitectura	Se ejecuta en el mismo loop que la app	Controla el navegador externamente
Lenguajes	JavaScript/TypeScript	Múltiples (Java, Python, C#, etc.)

Navegadores	Chrome, Firefox, Edge, Electron	Todos los navegadores principales
Velocidad	Más rápido (sin overhead de red)	Más lento (comunicación WebDriver)
Esperas	Automáticas e inteligentes	Manuales (explícitas/implícitas)
Curva de aprendizaje	Baja	Media-Alta

Caso de estudio

Caso: Plataforma de Comercio Electrónico - "TechStore"

Contexto: TechStore es una plataforma de venta de productos tecnológicos que necesita garantizar una experiencia de usuario óptima, con tiempos de carga rápidos, seguridad en transacciones y funcionalidad confiable.

Aplicación del Aseguramiento Integral:

Tabla 3 - Estrategia de Testing Implementada

Tipo de Prueba	Herramienta	Qué se Validó	Resultado
Unitarias	Jest	Funciones de cálculo de precios, descuentos, impuestos	95% cobertura
Integración	Supertest	APIs de carrito de compras, inventario, pagos	100% endpoints críticos
Rendimiento	k6	Tiempo de carga bajo 1000 usuarios concurrentes	< 2 segundos respuesta

Seguridad	OWASP ZAP	Vulnerabilidades XSS, SQL Injection, CSRF	0 vulnerabilidades críticas
E2E	Cypress	Flujo completo: búsqueda → carrito → checkout → pago	15 flujos críticos validados
Compatibilidad	BrowserStack	Funcionamiento en Chrome, Firefox, Safari, Edge	100% compatible
Usabilidad	Hotjar	Tasa de conversión, abandonos de carrito	40% reducción en abandonos

5. Conclusiones

El aseguramiento integral del software va más allá de verificar que las funciones trabajen correctamente; implica validar que el sistema sea rápido, seguro, escalable y usable. Las pruebas no funcionales son esenciales para garantizar la calidad de la experiencia del usuario y la viabilidad técnica a largo plazo. Las pruebas end-to-end, especialmente cuando se automatizan con herramientas modernas como Cypress, proporcionan confianza en que los flujos críticos del negocio funcionan correctamente en condiciones reales. Implementar una estrategia de testing balanceada, que combine pruebas unitarias, de integración, E2E y no funcionales, es fundamental para entregar software de alta calidad que satisfaga tanto las necesidades funcionales como las expectativas de rendimiento y confiabilidad.

6. Referencias

1. **Bach, J. & Bolton, M.** (2023). *Software Testing Heuristics*. <https://www.satisfice.com>
2. **Gleeson, G.** (2023). *Cypress Documentation*. <https://docs.cypress.io>
3. **ISTQB** (2023). *ISTQB Foundation Level Syllabus*. International Software Testing Qualifications Board. <https://www.istqb.org>
4. **Martin, R. C.** (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
5. **Newman, S.** (2021). *Building Microservices* (2nd ed.). O'Reilly Media.

6. **OWASP Foundation** (2023). *OWASP Testing Guide*.
<https://owasp.org/www-project-testing-guide>